Computer Organization and Assembly Language Lab

Project Report

Session: Fall 2024

Submitted By:

Muhammad Ayan Sajid	**2024-CS-661**
Muhammad Husnain	**2024-CS-696**
Fiza Shahid Khan	**2024-CS-680**
Shareen Asim	**2024-CS- 693**
Abdul Hadi	**2024-CS-655**

Submitted To:

Mr. Noman Munir

Department of Computer Science, UET Lahore, New Campus

Table of Contents

Table of Figures

1 Problem Statement

The objective of this Open Ended Lab (OEL) was to design and implement a menu-driven assembly language toolkit combining numeric data processing, string handling, and bit-level security operations. The system processes datasets, manages user interactions, and securely saves data using standard low-level programming concepts (registers, memory variables, arrays, string buffers, macros, and file I/O) via the MASM Irvine32 environment.

2 Program Architecture & Modular Design

To ensure high code quality and testability, the project implements a strict modular design across five files, utilizing PROTO and EXTERNDEF for linking:

- **globals.inc**: Acts as the shared header. Contains the Start_level and Reset_Str macros, global variables, and procedure prototypes.
- **main.asm**: Handles boot-sequence file loading, user menu interactions, and conditional branching.
- **auth.asm (Security Utility)**: Handles user authentication, bitwise encryption, and .txt file I/O.
- **hangman.asm (String Processor)**: Manages the 10-level word guessing game, string primitives, dynamic memory modifications, and string reversal.
- **results.asm (Numeric Analyzer)**: Calculates averages, maintains maximum scores, tracks history via DWORD arrays, and manages binary .dat file I/O.

3 Implementation of Assembly Language Concepts

3.1 Bit-Level & Security Operations

The authentication module encrypts passwords using a combination of XOR masking and Bitwise Rotations (ROL/ROR).

1 **Encryption:** XOR AL, 5Ah followed by ROL AL, 1.

2 **Decryption:** ROR AL, 1 followed by XOR AL, 5Ah.

The encrypted bytes are stored directly into password.txt, ensuring the raw password is never visible in the file system.

3.2 String Processing

- **Case Conversion:** User input is converted from lowercase to uppercase using the bitwise operation AND AL, 11011111b.

- **String Reversal:** If the user fails a level, the actual word is printed backwards using a loop with the ESI and EDI registers.
- **Memory Reset:** Between games, the macro Reset_Str utilizes the string primitive REP MOVSB alongside the CLD flag to rapidly copy clean template strings over the modified hidden string buffers.

3.3 Numeric & Data Processing

- **Averages & Limits:** The score calculation utilizes the MUL and DIV operations: Average = ((Avg * Rounds) + New Score) / (Rounds + 1).
- **Array Handling:** A session history array (score_history DWORD 100 DUP(0)) is populated using scaled-index addressing (score_history[EBX * 4]) to log up to 100 round results.

3.4 Input/Output & File Handling

The project implements persistent storage.

- Uses OpenInputFile, ReadFromFile, and WriteToFile APIs.
- High scores are securely stored as raw 32-bit registers (4 bytes) inside a binary file highscore.dat.

4 Register & Memory Usage

Register	***Primary Usage in Project***
EAX	Standard I/O (ReadInt/WriteInt), math accumulator, returning File Handles.
EBX	Array index scaling (EBX * 4) and storing File Handles during I/O.
ECX	Loop counters and tracking the exact number of missing letters in strings.
EDX	String offset pointers for WriteString and CreateOutputFile.
ESI / EDI	Source and Destination pointers for string arrays and REP MOVSB macros.

5 Program Flow (Flowchart summary)

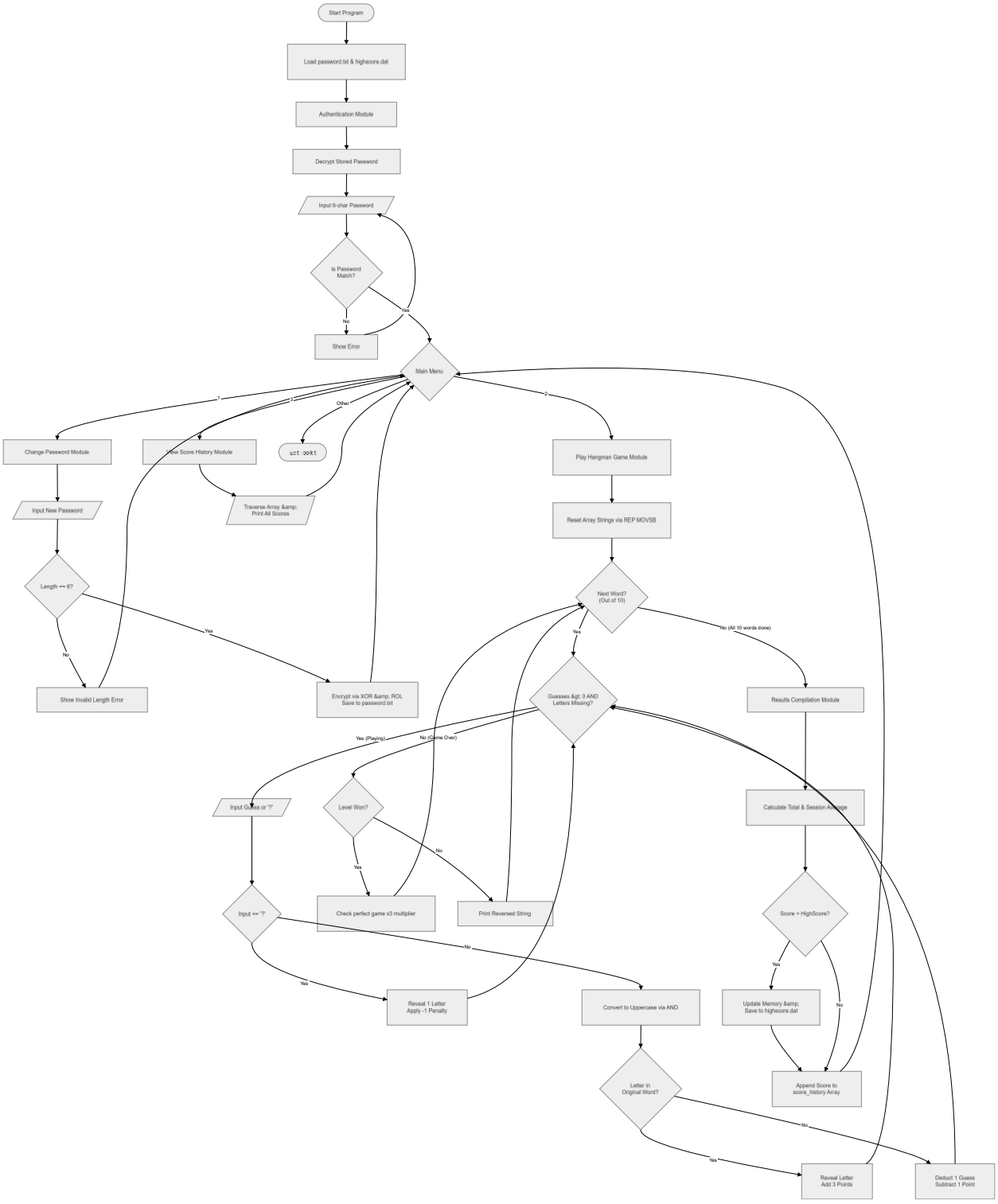

Figure 1 Flowchart

1. **Boot:** Program loads password.txt and highscore.dat into .data memory.
2. **Auth:** Prompts user for 6-character password. Compares using REPE CMPSB.
3. **Menu:** Allows selection between (1) Change Password, (2) Play Game, (3) View History, (4) Exit.
4. **Game Loop:** Loads template strings. Takes character input -> Bitwise uppercase conversion -> Array search -> Updates hidden string -> Updates numeric score.
5. **Post-Game:** Checks if Score > High_Score. Saves to file. Appends score to score_history array. Returns to Menu.

6 Test Cases & Debugging Evidence

Test Case	*Input*	*Expected Output*	*Actual Result*
Invalid Auth	"999999"	"Incorrect Password! Try again."	Passed. Denied entry.
Valid Auth	"123456"	"Authentication Successful!" & Menu opens.	Passed.
Change Password Length	"123"	"Invalid password! Must be exactly 6 characters."	Passed. Handled boundary case.
String Case Handling	Guess lowercase 'a'	Safely converts to 'A', matches array, reveals letter.	Passed.
Hint Feature ('?')	Input '?'	Deducts 1 pt, removes x3 multiplier, reveals 1 letter.	Passed.
Level Failure	5 wrong guesses	Shows "Failed" and prints word reversed (String Reversal).	Passed.
Score History Array	Press '3' in Menu	Prints "Game 1: [Score]", "Game 2: [Score]".	Passed. Array traversal correct.
Data Persistence	Restart App	High score and changed password persist perfectly.	Passed. File I/O operational.

6.1 Output Screenshots

Figure 2 Authentication & Main Menu interface

```
Select an option: 2

Current Word: A__EMB_Y
Wrong guesses left: 5
Enter a letter to guess (or '?' for Hint): s

Current Word: ASSEMB_Y
Wrong guesses left: 5
Enter a letter to guess (or '?' for Hint):

Current Word: ASSEMB_Y
Wrong guesses left: 4
Enter a letter to guess (or '?' for Hint): ?
>>> HINT USED! Revealing 1 letter (-1 Point).

Current Word: ASSEMBLY
Excellent! You cleared the level.
Level Score (Added): +4

Current Word: R_GI_T_R
Wrong guesses left: 5
Enter a letter to guess (or '?' for Hint): r

Current Word: R_GI_T_R
Wrong guesses left: 4
Enter a letter to guess (or '?' for Hint):
```

Figure 3 Playing the game (showing character replacement and hints)

```
--- GAME OVER: ROUND RESULTS ---
Total Score for this round: +31
All-Time Highest Score: +51
Session Average Score: +31
Session Rounds Played: 1
```

Figure 4 Game Over screen (showing Math/Average calculations)

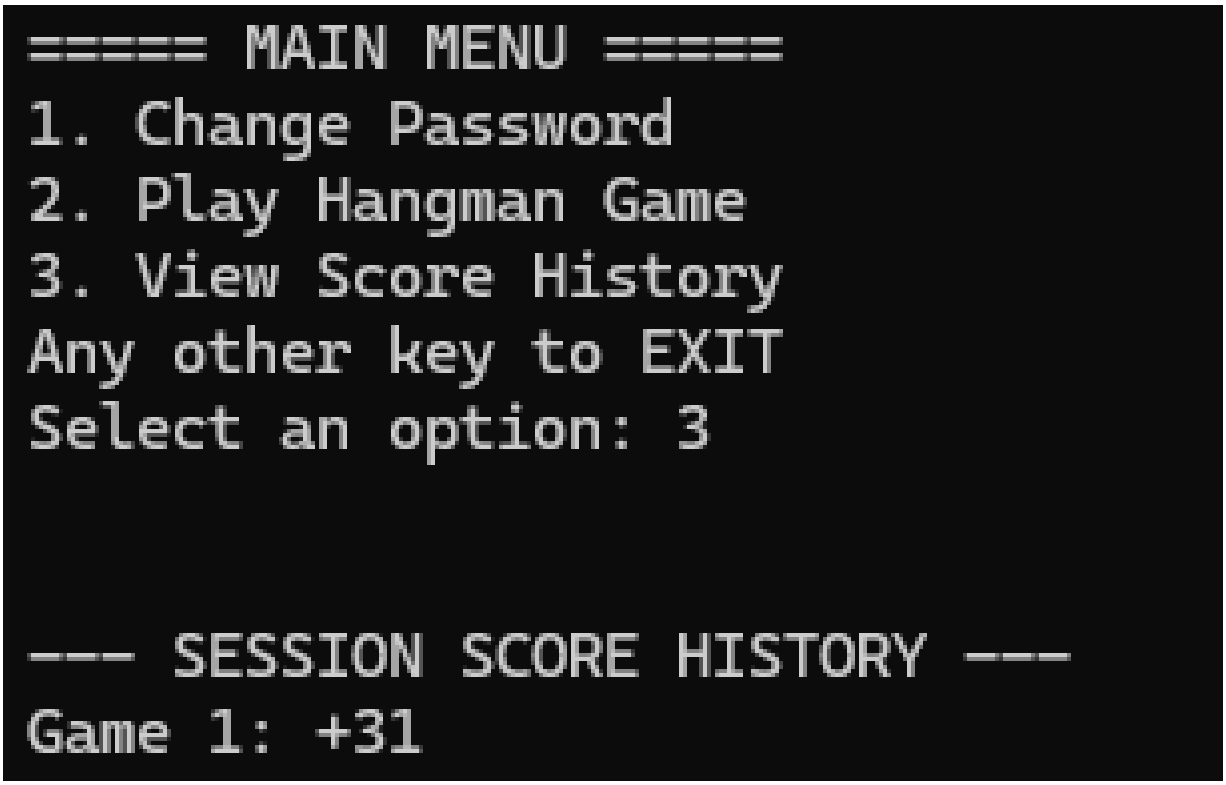

Figure 5 Array Traversal (Option 3: View Score History)

7 Conclusion

This project successfully fulfills the requirement of the CSC205L Open Ended Lab by deploying an advanced, fully modular Assembly Language application. It heavily utilizes low-level data tracking, memory buffering, bitwise security, numeric mathematics, and file streaming to create a stable, crash-free user experience.